Computación en Internet III
Ingeniería de Sistemas
Facultad Barberi de Ingeniería, Diseño y Ciencias Aplicadas

Parcial NestJS (Corte 2): Computación en Internet III
Docente: Gustavo Adolfo González Medina
Entrega: jueves, 15 de octubre de 2026
Tiempo para desarrollarlo: 2 horas
Repositorio: El proyecto estará disponible en la publicación de este parcial, al mismo nivel de este documento, en formato zip.

Condiciones del examen:

- El estudiante podrá visualizar sus apuntes del curso, así como la documentación del curso y enlaces relacionados al contexto del ejercicio. Podrá hacer uso de internet y búsquedas de código a través del navegador.
- **El estudiante no podrá hacer uso de ningún tipo de herramienta relacionada con IA Generativa como lo son: Chat GPT, Gemini, Copilot, Claude y similares.** De ser utilizado se anulará el examen y se hará el respectivo reporte para iniciar un proceso disciplinario.
- El estudiante no podrá hacer uso de ningún dispositivo electrónico además de su computador personal o el suministrado por el laboratorio para realizar la evaluación.
- El estudiante deberá hacer uso de las herramientas utilizadas dentro del curso hasta el momento, especialmente **NestJS, TypeORM, PostgreSQL, JWT, TypeScript y Docker**.
- Dentro del examen, el docente le solicitará hacer commits a su repositorio de manera frecuente, esto con el objetivo de demostrar su desarrollo y avance a través del examen. Si su entrega final no posee un avance demostrable, su entrega será revisada posteriormente para demostrar su autenticidad.
- **No modifique el comportamiento de los módulos existentes** (`student`, `user` y `seed`). Solo agregue código nuevo; la única excepción es la relación nueva en `Student`.

Nota final: Si la aplicación falla al ejecutarse, su nota podrá verse afectada drásticamente. Además, si el profesor lo considera, el examen deberá ser sustentado para validar sus conocimientos.

Advertencia: Haga commit y push a su repositorio de GitHub Classroom antes de que termine el tiempo de la evaluación. Se revisará el **último commit dentro del horario asignado**. **No suba su archivo .env**.

Objetivo de la asignación:

Se le entrega una API REST construida con **NestJS 11**, **TypeORM** y **PostgreSQL**. Es el mismo proyecto trabajado en clase: los módulos `student` (con `grades`), `user` (registro, login, JWT, `@Auth()`, `@GetUser()`) y `seed` **ya están implementados y funcionando**. Úselos como referencia del estilo de código que se espera.

Su trabajo es construir un módulo nuevo, `program` (programa académico, por ejemplo "Ingeniería de Sistemas"), completando los **4 retos** marcados con `// TODO` en el código.

Requisitos previos

- Node.js 18+
- PostgreSQL accesible (local o Docker)

Instalación y ejecución

1. Instale las dependencias: `npm install`
2. Copie `.env.example` como `.env` y ajuste los valores de su base de datos: `cp .env.example .env`. Si no tiene un PostgreSQL corriendo, puede levantar uno con Docker:

```
docker run -d --name postgres-icesi \
  -p 5432:5432 \
  -e POSTGRES_PASSWORD=postgres \
  -e POSTGRES_DB=compunet3 \
  postgres:14.3
```

3. Levante la app: `npm run start:dev`
4. Cargue los 40 estudiantes de prueba: `GET http://localhost:9000/api/seed`

La API queda en `http://localhost:9000/api` (la documentación Swagger se sirve en esa misma URL desde el navegador). Importe la colección de Postman `postman/2026-2-evaluacion-corte2.postman_collection.json`: trae una petición de ejemplo para cada reto.

Cómo obtener un usuario *admin*

Todo usuario nuevo se registra con el rol `teacher`. Para probar los endpoints protegidos del Reto 4 necesita un `admin`:

1. Regístrese con `POST /api/user/signup`.
2. Asígnele el rol `admin` directamente en la base de datos:

```
UPDATE "user" SET roles = '{admin}' WHERE email = 'tu-correo@icesi.edu.co';
```

3. Inicie sesión con `POST /api/user/auth` y use el `token` en el header `Authorization: Bearer <token>`.

Dónde trabajar

Busque los comentarios `// TODO` en:

- `src/program/`: `program.module.ts`, `program.controller.ts`, `program.service.ts`, `entities/program.entity.ts`, `dto/create-program.dto.ts`, `dto/assign-students.dto.ts`, `dto/bulk-create-program.dto.ts`
- `src/student/entities/student.entity.ts`: la relación con `Program`
- `src/app.module.ts`: registrar el módulo

Use **imports relativos** (`../../student/entities/student.entity`) en lugar de `src/...`: Jest no resuelve ese alias y `npm run test` dejaría de pasar.

Reto 1 — Módulo y entidad Program (20 pts)

`POST /api/program` · `GET /api/program`

Cree la entidad `Program` (tabla `program`) y su módulo:

Columna	Tipo	Reglas
`id`	UUID	Generado automáticamente
`name`	text	Único
`code`	text	Único
`credits`	int	—

Body de ejemplo (POST):

```
{ "name": "Ingenieria de Sistemas", "code": "SIS", "credits": 160 }
```

Criterios de aceptación:

- ☐ `ProgramModule` está registrado en `AppModule` y la entidad `Program` con `TypeOrmModule.forFeature`. Al arrancar, se crea la tabla `program`.
- ☐ `CreateProgramDto` valida: `name` string de mínimo 3 caracteres; `code` de 3 a 6 letras **mayúsculas**; `credits` entero positivo. Si algo no cumple, la respuesta es `400`.
- ☐ `POST /api/program` crea el programa y responde `201` con el programa creado.
- ☐ Un `name` o `code` repetido responde `409 Conflict`, no `500` ni una respuesta vacía.
- ☐ `GET /api/program` lista los programas y acepta `limit` y `skip` (reutilice `PaginationDto`).

Reto 2 — Relación con Student y asignación acumulativa (20 pts)

`GET /api/program/:id` · `PATCH /api/program/:id/students`

Un programa tiene **muchos** estudiantes; un estudiante pertenece a **un** programa (o a ninguno).

El `PATCH` **agrega** estudiantes al programa: los que ya estaban asignados se conservan. Resuélvalo con el mismo patrón del `update` de estudiantes corregido en clase, el que combina las notas actuales con las nuevas en vez de reemplazarlas:

1. Dentro de la transacción, traiga el programa con los estudiantes que tiene hoy.
2. Combine: agregue solo los estudiantes nuevos (los que ya están no se repiten).
3. Guarde la lista completa con `queryRunner.manager.save(program)`.

> **Ojo:** si guarda `program.students` solo con los estudiantes nuevos, TypeORM deja a los anteriores **sin programa** (`programId` en `NULL`). Es el mismo problema de las notas huérfanas que vimos en clase con el `update` de estudiantes.

Body de ejemplo (PATCH):

```
{ "studentIds": ["<uuid estudiante 1>", "<uuid estudiante 2>"] }
```

Criterios de aceptación:

- ☐ Relación `@ManyToOne` en `Student` (propiedad `program`, opcional) y `@OneToMany` en `Program` (propiedad `students`). En la tabla `student` aparece la columna `programId`.
- ☐ `AssignStudentsDto` valida que `studentIds` sea un arreglo **no vacío** de UUIDs. Si no, `400`.
- ☐ `GET /api/program/:id` devuelve el programa **con sus estudiantes**. Si el `id` no es un UUID, `400`; si no existe, `404`.
- ☐ El `PATCH` usa una **transacción** (`QueryRunner`: `connect`, `startTransaction`, `commitTransaction` / `rollbackTransaction`, `release`) y responde `200` con el programa y sus estudiantes.
- ☐ Es **acumulativo**: asignar `[A, B]` y luego `[C]` deja al programa con `A`, `B` y `C`. Enviar un estudiante que ya está asignado no lo duplica ni da error.
- ☐ Si el programa no existe: `404`.
- ☐ Si **algún** `studentId` no existe: `404` y **ningún** estudiante de esa petición queda asignado (rollback).

Reto 3 — Creación de programas en lote (15 pts)

`POST /api/program/bulk`

Cree varios programas en una sola petición, con la regla **todo o nada**: si alguno falla, no se crea ninguno.

Body de ejemplo:

```
{
  "programs": [
    { "name": "Arquitectura", "code": "ARQ", "credits": 170 },
    { "name": "Derecho", "code": "DER", "credits": 165 }
  ]
}
```

Criterios de aceptación:

- ☐ `BulkCreateProgramDto` valida que `programs` sea un arreglo **no vacío** y que **cada** elemento cumpla las reglas de `CreateProgramDto` (validación anidada). Un solo elemento inválido → `400`.
- ☐ Si todo es válido, responde `201` con los programas creados.
- ☐ Si algún `name` o `code` ya existe en la base de datos, **o** se repite dentro del mismo lote: `409` y **no se crea ninguno** (transacción).

Reto 4 — Seguridad (15 pts)

Criterios de aceptación:

- ☐ `POST /api/program`, `POST /api/program/bulk` y `PATCH /api/program/:id/students` solo los puede usar un usuario con rol `admin`: sin token, `401`; con un usuario `teacher`, `403`.
- ☐ `GET /api/program` y `GET /api/program/:id` siguen siendo públicos.
- ☐ `Program` tiene una relación `@ManyToOne` con `User` (propiedad `createdBy`). Al crear programas (uno o en lote) se guarda el admin que los creó, obtenido del token con `@GetUser()`, **no** del body.

Rúbrica:

Apartado	Puntos
Reto 1 — Módulo y entidad Program	20
Reto 2 — Relación con Student y asignación acumulativa	20
Reto 3 — Creación en lote	15
Reto 4 — Seguridad	15
Cuestionario de selección múltiple (se presenta por separado)	30
Total	**100**

La nota sobre 5.0 se calcula como **puntos / 20**. Si el proyecto no compila o no arranca, se calificará revisando el código, con máximo el 50 % de cada criterio que esté bien planteado.

Reto 1 — Módulo y entidad Program

Criterio	Puntos
Entidad `Program` con `@Entity()`, `id` UUID generado, `name` y `code` únicos, `credits` int	4
`ProgramModule` registrado en `AppModule` y `TypeOrmModule.forFeature([Program])` en el módulo	3
`CreateProgramDto` con las validaciones de `name`, `code` y `credits`; datos inválidos → `400`	5
`POST /api/program` crea y responde `201` con el programa	3
Nombre o código repetido → `409 Conflict`	3
`GET /api/program` con paginación (`PaginationDto`)	2
Total	**20**

Reto 2 — Relación con Student y asignación acumulativa

Criterio	Puntos
Relaciones `@ManyToOne` / `@OneToMany` con las funciones inversas correctas; columna `programId` en `student`	4
`AssignStudentsDto`: arreglo no vacío de UUIDs; si no, `400`	2
`GET /api/program/:id` con sus estudiantes; id no UUID → `400`; inexistente → `404`	4
Uso correcto de `QueryRunner`: `connect`, `startTransaction`, operaciones con `queryRunner.manager`, `commitTransaction`, `rollbackTransaction` en el `catch` y `release` en el `finally`	4
Asignación **acumulativa**: conserva los anteriores y no duplica	4
Atomicidad: `studentId` inexistente → `404` y ninguno asignado; programa inexistente → `404`	2
Total	**20**

Reto 3 — Creación en lote

Criterio	Puntos
`BulkCreateProgramDto`: arreglo no vacío con validación anidada de cada elemento	5
`POST /api/program/bulk` crea todos y responde `201` con los programas creados	3
Todo o nada: nombre/código ya existente o repetido dentro del lote → `409` y no se crea ninguno	7
Total	**15**

Reto 4 — Seguridad

Criterio	Puntos
Endpoints de escritura solo para `admin`: sin token → `401`, `teacher` → `403`; los `GET` siguen públicos	5
`ProgramModule` importa lo necesario para que la autenticación funcione	2
`Program` tiene `@ManyToOne(() => User)` en `createdBy`	3
`createdBy` se llena con `@GetUser()` (en el POST y en el bulk), no desde el body	5
Total	**15**